SOFTWARE ENGINEERING · ASSIGNMENT #04

# AI-assisted evolution of the Smart Campus Room Availability Checker

Refactoring, room-catalog feature extension and automated quality evidence


| | |
|---|---|
| Full name | **Yeraliyeva Anel** |
| Student ID | **24B031020** |
| Practice lessons | **Tuesday, 18:00 – 19:00** |
| AI assistant used | **Claude (Anthropic), model Claude Opus 5.5** |
| Date | 8 November 2026 |

STARTING POINT AND SCOPE

# Baseline: reconstructed single-room prototype

Assignment #03 was not completed, so — as the brief allows — an **equivalent minimal version** of the single-room prototype was reconstructed first and committed as the baseline ( `48bd593` ), so every later change has a recorded starting point.

- `baseline/app.js` — 94 lines: room data, rules and DOM handlers together
- One room (*Room 301*), two fictional bookings
- Fixed current time **2026-11-10 09:00**
- No login, payments, live APIs or database
- Baseline AC regression suite: **8 / 8 passed**

| ID | Acceptance criterion (preserved) |
|---|---|
| AC1 | Valid HH:MM, end after start, not in the past |
| AC2 | Max 120 minutes (120 ok, 121 rejected) |
| AC3 | Blocked room cannot be booked |
| AC4 | Overlap rejected, touching intervals allowed |
| AC5 | Booking data is never mutated |
| AC6 | Clear, responsive feedback |

TASK 01 · CODEBASE ANALYSIS

# Structured context given to the AI assistant

## Repository map (as provided)

```
baseline/
  index.html     form: date, start, end
  app.js         room data, NOW, checkAvailability(),
                 submit + change handlers
  tests/         AC1–AC5 regression checks
```

## Constraints stated explicitly

- Fictional data only; no login / payments / APIs / DB
- Do not change AC1–AC6 behaviour
- Small plan first, one change at a time — no full rewrite

## Requirements stated explicitly

- AC1–AC6 (previous slide), with boundary values
- New: ≥ 4 rooms (id, name, capacity, equipment, blocked, bookings)
- F1 capacity ≥ attendees · F2 ALL equipment · F3 blocked never available · F4 empty state · F5 room switch refreshes result
- ≥ 10 executable tests incl. 120/121 min, touching/overlap, exact/insufficient capacity, one UI check

## Source of the context

The assignment brief (Lesson 08 slides) and the submission instructions were given verbatim. Because there was no Assignment #03 code, the assistant first reconstructed the minimal baseline from AC1–AC6; that file was then the codebase it analysed.

TASK 01 · CODEBASE ANALYSIS

# Maintainability risks found in the baseline

| # | Risk | Evidence (file / function / line) | Why it matters for this change |
|---|---|---|---|
| R1 | **Hard-wired single room** | `app.js:6` global `room`; `checkAvailability()` line 16 `r = r \|\| room` | Silent fallback to Room 301 — exactly the "checked another room's data" bug the new feature must avoid |
| R2 | **Duplicated time validation** | Same HH:MM regex in `checkAvailability()` l.18, submit handler l.65, change handler l.80 | Three places to update; message text also copied 2× |
| R3 | **Duplicated result rendering** | `out.className = res.ok ? …` at l.71 and l.86 | New states (blocked, empty, no room) would need edits in both handlers |
| R4 | **Mixed responsibilities** | Data (l.3–13), rules (l.15–52) and DOM (l.55–91) in one file | Rules cannot be tested without guarding for `document`; UI cannot be tested at all |
| R5 | **Mutable booking data** | `room.bookings` is a plain array (l.9) | AC5 holds only by convention; any future code could `push` into it |

TASK 01 · CODEBASE ANALYSIS

# Prioritised plan (agreed before editing) and a rejected suggestion

| P | Step | Fixes |
|---|---|---|
| 1 | Freeze baseline in Git + AC regression tests | safety net |
| 2 | Extract `time.js` (parse, overlap) | R2 |
| 3 | `availability.js`: room passed explicitly, no default | R1, R4 |
| 4 | `rooms.js` catalog, deep-frozen | R1, R5 |
| 5 | `filters.js` for F1–F3 | new |
| 6 | `render.js` + `app.js` controller | R3, R4 |
| 7 | Run all tests + review diff after each step | — |

**Rejected: "hide blocked rooms from the list"**

The simplest way to satisfy F3 was to filter blocked rooms out entirely. Rejected: a user looking for the Main Auditorium would not know *why* it is missing. Instead the room stays visible with a **BLOCKED** label, and F3 is enforced where it matters — `checkAvailability()` returns `reason: "blocked"`. Covered by test *AC3/F3*.

**Modified: input handler in `ui/app.js`**

The first generated handler re-validated the slot before deciding whether to refresh — a second copy of AC1 logic, re-creating risk R2. On diff review it was reduced to a single `refresh()` call; all 25 tests still pass.

TASK 02 · FEATURE EXTENSION

# Room catalog and filtering rules

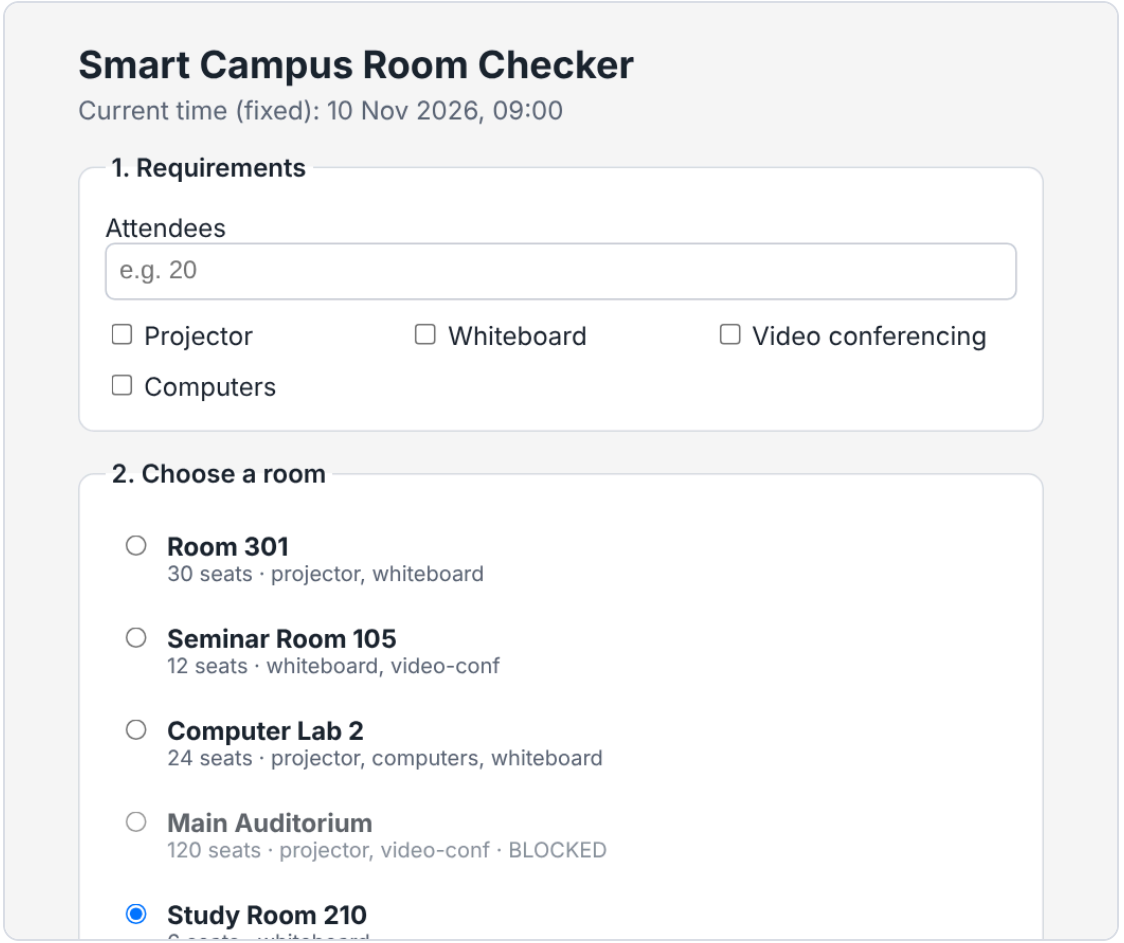
| ID | Name | Cap. | Equipment | Blocked | Bookings (10 Nov) |
|---|---|---|---|---|---|
| R301 | Room 301 | 30 | projector, whiteboard | no | 10:00–11:00, 13:00–14:30 |
| R105 | Seminar Room 105 | 12 | whiteboard, video-conf | no | 11:00–12:00 |
| LAB2 | Computer Lab 2 | 24 | projector, computers, whiteboard | no | 09:00–10:30 |
| AUD1 | Main Auditorium | 120 | projector, video-conf | **yes** | — |
| R210 | Study Room 210 | 6 | whiteboard | no | — |

All data fictional, defined in `final/src/data/rooms.js`, deep-frozen with `Object.freeze`.

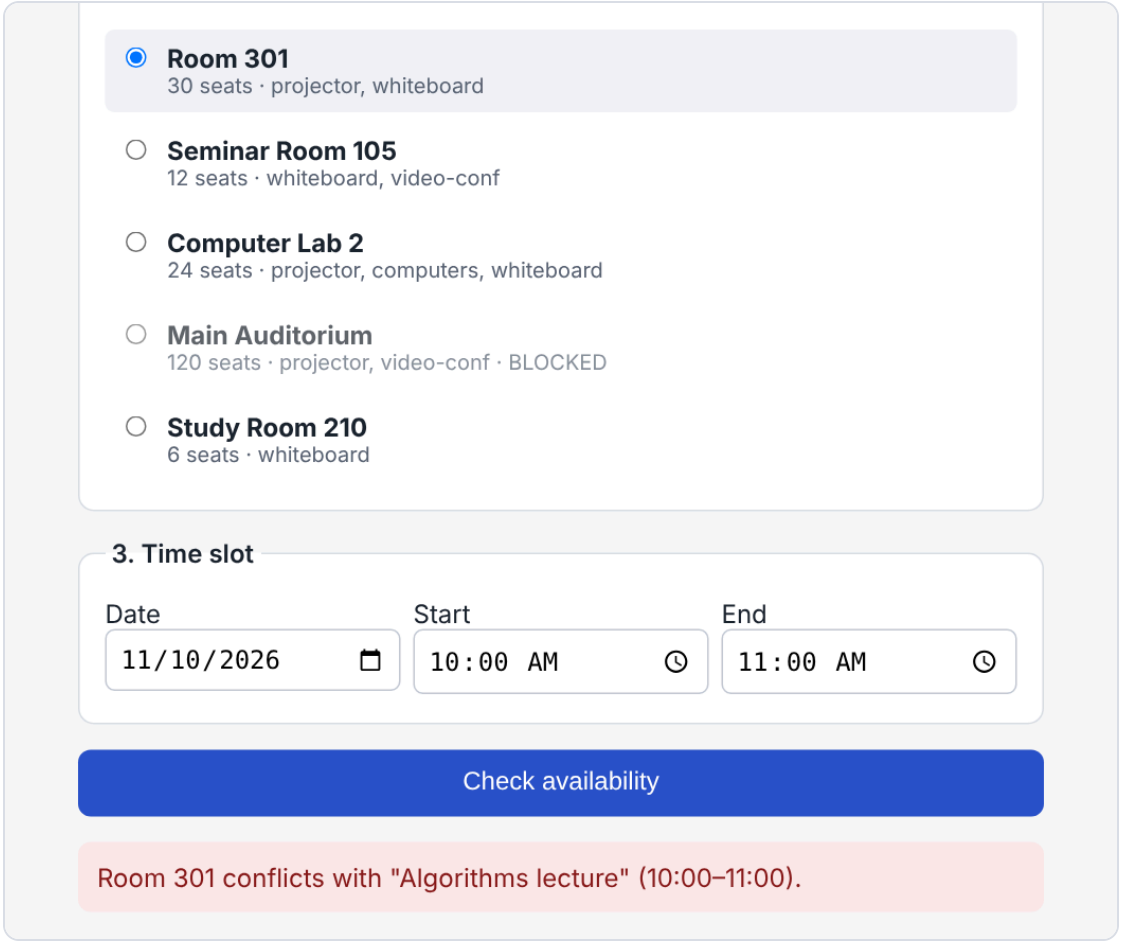
- **F1** `meetsCapacity()`: `capacity >= attendees`; empty input = no requirement
- **F2** `hasAllEquipment()`: `required.every(...)`
- **F3** blocked rooms listed but flagged; `checkAvailability()` never returns ok for them
- **F4** "No rooms match these requirements..." message; selection and result cleared
- **F5** every room change calls `refresh()`, which re-checks with *that* room's bookings via `findRoom()`
- AC1–AC4 logic unchanged in meaning, now in `validateSlot()` + `checkAvailability(room, slot, now)`

TASK 02 · DEMONSTRATION

# Successful check, then switching room (F5)



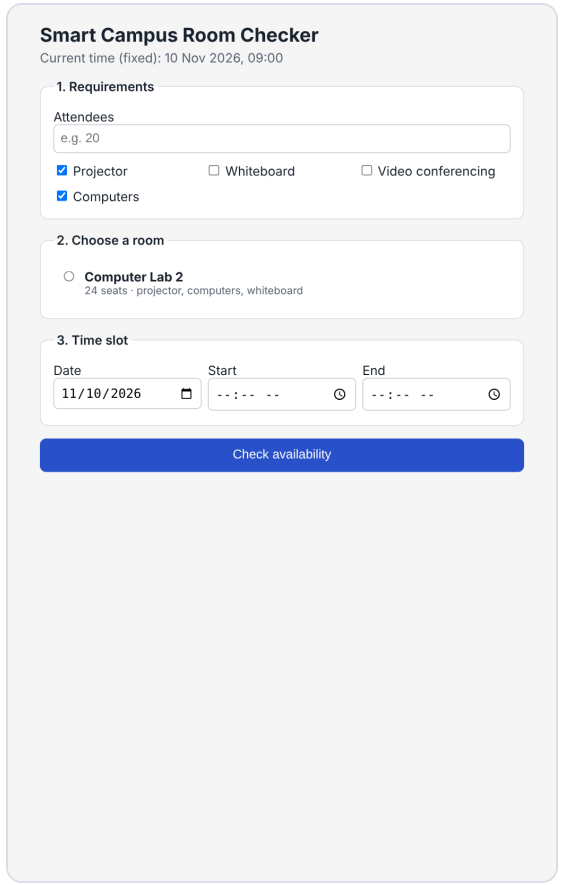

Study Room 210, 10:00–11:00 → available (green).



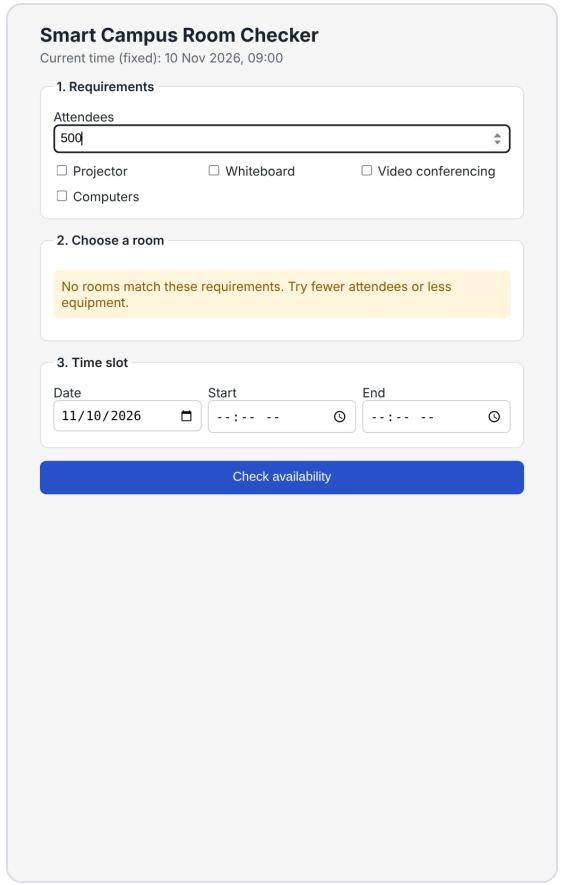

Same slot, switched to Room 301 → result refreshes automatically: conflict with "Algorithms lecture".

TASK 02 · DEMONSTRATION

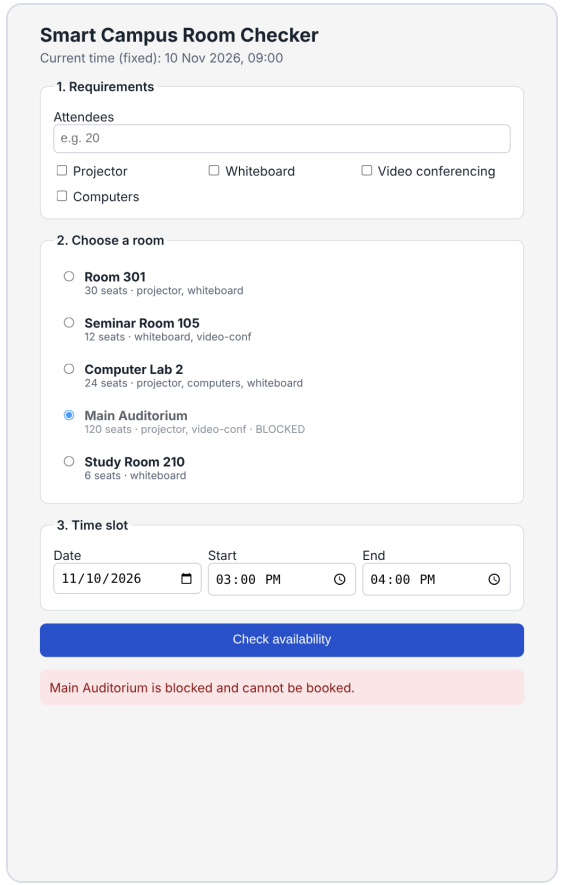
# Filtering, empty state, blocked room, 121 minutes on mobile



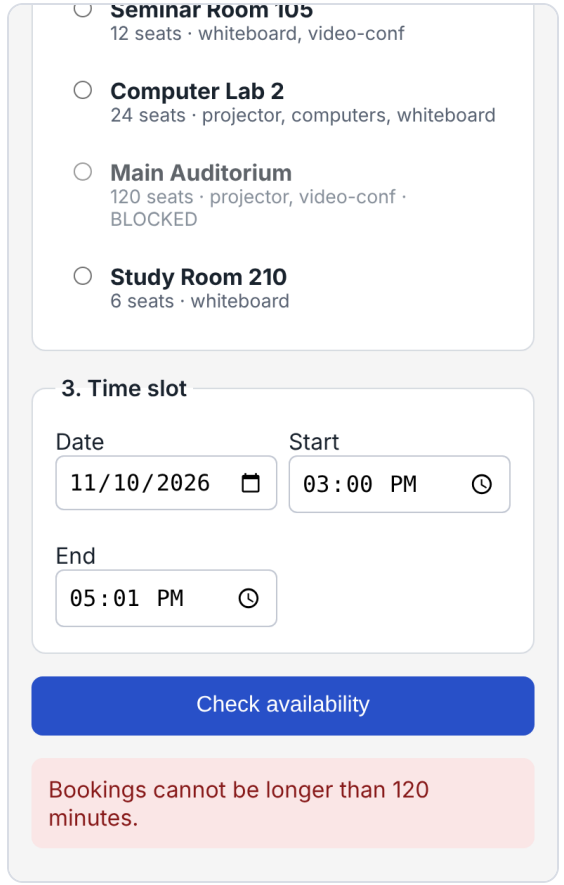

F2: projector + computers → only Computer Lab 2.




F4: 500 attendees → clear empty-state message.




F3: Main Auditorium, free slot → still "blocked".

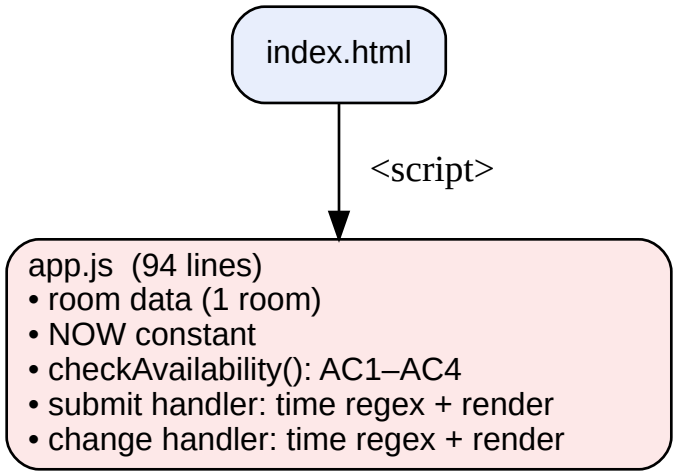



AC2/AC6: 15:00–17:01 at 390 px width → error, layout reflows.

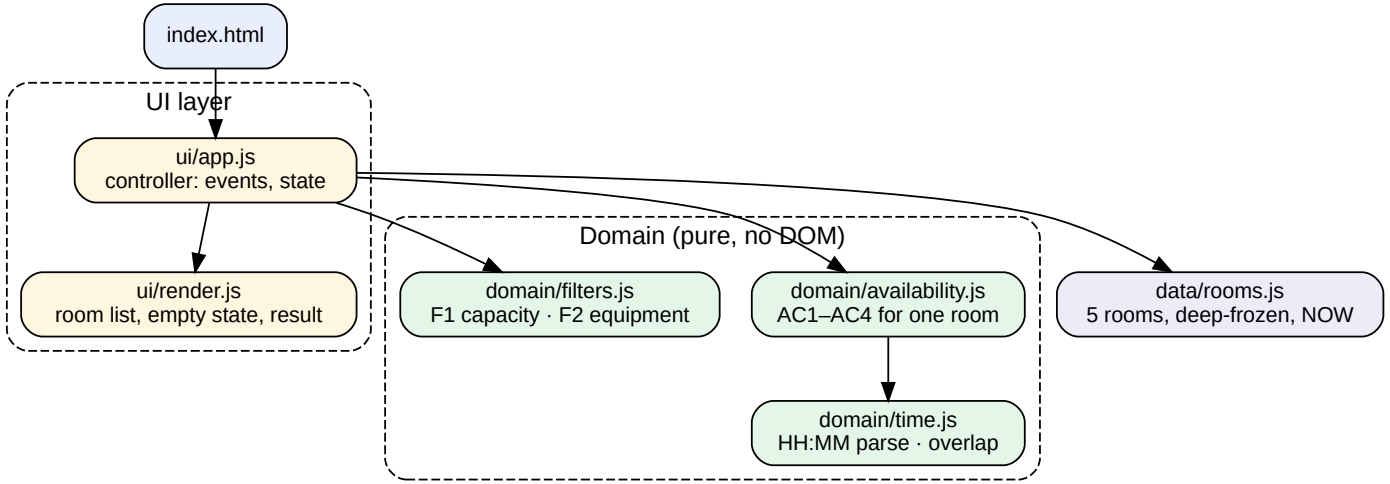
TASK 03 · REFACTORING

# Before / after module diagram

**Before** — 1 module




**After** — 6 modules in 3 layers



TASK 03 · REFACTORING

# Design decisions and their justification

### D1. Pure domain layer, room passed explicitly

`checkAvailability(room, slot, now)` has no default room and no DOM access. The baseline fallback `r = r || room` (R1) is gone, so it is impossible to check a slot against another room's bookings by accident. Purity also lets 20 of the 25 tests run in under 2 ms each without a browser.

### D2. Single time module with half-open intervals

`time.js` owns HH:MM parsing and `overlaps(a,b) = aStart < bEnd && bStart < aEnd`. One definition replaces three regex copies (R2) and makes the touching-vs-overlap rule explicit. A mutation to `<=` is caught by the AC4 boundary test.

### D3. Deep-frozen catalog

AC5 is now enforced by the runtime instead of convention (R5): any write to a booking throws `TypeError` in strict mode, asserted in the AC5 test.

### D4. One render path, controller owns state

`render.js` has one `renderResult()` (R3). `app.js` keeps `selectedId`; when filters hide the selected room it is cleared, so a stale result is never shown (F4/F5).

TASK 04 · AUTOMATED TESTING

# 25 executable tests — Node built-in test runner + jsdom

| File | Tests | Covers |
|---|---|---|
| `availability.test.js` | 12 | AC1 (malformed, end<start, past) · **AC2 120 vs 121 min** · AC3/F3 blocked · **AC4 touching 11:00 vs overlapping 10:59**, containing · AC5 frozen & unchanged · same slot busy in R301 / free in R210 · no room selected |
| `filters.test.js` | 8 | catalog shape (≥4 rooms, all fields) · **F1 exact capacity 12 vs 13** · empty attendees · F2 ALL items (2 cases) · F1+F2 combined · F4 empty list |
| `ui.test.js` | 5 | **Integration/UI** on real `index.html` in jsdom: success feedback · **F5 room switch refresh** · F4 empty state · F1/F2 list update · AC6 error text + viewport |
| `baseline.test.js` | 8 | AC1–AC5 against the original `baseline/app.js` (regression reference) |

## Commands actually run (8 Nov 2026, Node v22.23.4)

```
$ npm install                 # jsdom@24 dev dependency
$ npm run test:baseline       # node --test --test-reporter=spec baseline/tests/*.test.js
$ npm test                    # node --test --test-reporter=spec final/tests/*.test.js
```

# Results and checking that the tests can fail

```
✔ AC2 boundary: exactly 120 minutes is allowed
✔ AC2 boundary: 121 minutes is rejected
✔ AC4 boundary: touching interval 11:00–12:00 … allowed
✔ AC4 boundary: overlapping interval 10:59–11:30 rejected
✔ F1 boundary: exact capacity (12) keeps Seminar Room 105
✔ F1 boundary: insufficient capacity (13) removes …
✔ F5 (UI): switching room refreshes result …
✔ F4 (UI): clear empty state when no rooms match …
  … (25 lines total, see docs/evidence/)
ℹ tests 25
ℹ pass 25
ℹ fail 0
```

Baseline suite: **8 / 8** · Final suite: **25 / 25**

## Mutation checks (deliberate bugs, then reverted)

| Injected bug | Result |
|---|---|
| `overlaps()`: `<` → `<=` (touching = overlap) | **24 pass / 1 fail** — AC4 touching test |
| `hasAllEquipment()`: `every` → `some` | **16 pass / 9 fail** — F1, F2 and UI tests |

Both bugs were detected, so the suite checks behaviour rather than just running code. Output saved in `docs/evidence/mutation1.txt`, `mutation2.txt`.

## Problem found while testing

`node --test baseline/tests/` failed on Node 22 ("Cannot find module") — a directory is not accepted. Fixed with a file glob in `package.json`.

TASK 05 · QUALITY EVALUATION

# Original vs final — evidence table

| Measure | Original (baseline) | Final | How it was obtained |
|---|---|---|---|
| Automated tests passed / total | 8 / 8 (AC1–AC5 only) | 25 / 25 (AC1–AC6, F1–F5, UI) | `npm run test:baseline`, `npm test`; summary lines in `docs/evidence/*-output.txt` |
| Duplicated logic locations removed | HH:MM check ×3, result render ×2 | ×1 and ×1 → **3 removed** | `grep -c '\d{2}:\d{2}' baseline/app.js` = 3; `grep -c 'out.className = res.ok'` = 2; final `grep -rc 'HHMM.test\|className = `result'` = 1 each |
| Code structure | 1 module (94 lines): data + rules + DOM | 6 modules (194 lines): data · time · availability · filters · render · controller | `wc -l`; responsibilities from the module diagram (slide 9) |
| UI behaviour testable? | No (handlers inside `DOMContentLoaded`) | Yes — `mountApp(document)`, 5 jsdom tests | Presence of executable UI tests |
| Remaining limitations | • Single fixed date/time; bookings cannot be created, only checked · • Equipment names are free strings — no validation that a room's list uses the `EQUIPMENT` vocabulary · • Times use the browser's local time zone · • UI tests use jsdom, not a real browser (layout only checked manually via screenshots) · • No overnight (cross-midnight) slots | | |

Line counts are a size indicator, not a quality score: the final code is larger because it implements five new requirements.

TASK 06 · AI REFLECTION

# Reflection on working with the AI assistant

**Context.** I had not completed Assignment #03, so I first asked Claude to reconstruct a minimal single-room baseline from the acceptance criteria and committed it. I then gave it the full Assignment #04 brief, the submission instructions and that baseline, with the acceptance criteria AC1–AC6, the new rules F1–F5 and the constraints: fictional data only, no login, APIs or database, and no full rewrite. Stating the boundary values explicitly (120/121 minutes, touching vs overlapping, exact vs insufficient capacity) produced tests aimed at the risky edges instead of generic happy paths.

**Useful and incorrect suggestions.** The most useful output was the risk list, especially the hidden default room in `checkAvailability()`, which would have let the multi-room version check the wrong room's bookings. Not everything was right: the first input handler duplicated slot validation, recreating the problem the refactor was meant to remove, and the first test script used a directory path that Node 22 rejects. I also rejected hiding blocked rooms, because users would not understand why a room disappeared.

**What I verified.** I ran both test suites and read the output, injected two deliberate bugs to confirm the tests fail, opened the app and checked each scenario on the screenshots, and recounted the duplication metrics with grep instead of trusting estimates.

**Influence of tests.** Writing the baseline regression suite before refactoring meant every structural change could be accepted or rejected on evidence. The mutation checks showed the AC4 boundary test is the only guard for touching intervals, so I kept it as a named boundary case.

**Next.** I would add real-browser end-to-end tests, validate equipment names against one vocabulary, and support creating bookings with immutable updates.

AI-USE RECORD

# AI evidence appendix

| Tool / date | Prompt summary | Output used | Problems found | Changes made | Verification |
|---|---|---|---|---|---|
| Claude Opus 5.5 (Anthropic), claude.ai · 8 Nov 2026 | No Assignment #03 code: asked to reconstruct a minimal single-room baseline (AC1–AC6), then for risk analysis and a small plan | Baseline `app.js` + 8 AC tests; risks R1–R5; 7-step plan | Plan initially hid blocked rooms | Kept blocked rooms visible, flagged | Baseline 8/8 tests; line refs checked in `baseline/app.js` |
| same session | Implement catalog + F1–F5 one module at a time, keep AC1–AC6 | `rooms.js, time.js, availability.js, filters.js, render.js, app.js` | Input handler duplicated AC1 validation | Simplified to single `refresh()` | 25/25 tests; diff review; screenshots |
| same session | Write ≥10 tests incl. listed boundaries + one UI check | 3 test files, 25 tests; baseline suite 8 tests | Test script path failed on Node 22 | Glob in `package.json` | Commands re-run, output saved |
| same session | Confirm tests detect real defects | Two mutation runs | — | Mutations reverted | 24/25 and 16/25 when mutated; 25/25 after revert |
| same session | Draft README and this report | README, slides, diagrams (Graphviz) | — | Reviewed and edited | Metrics recomputed with `grep` / `wc` |

Verification sources: test runner output ( `docs/evidence/` ), Git history ( `48bd593 → a78e2e9` ), headless-Chrome screenshots, Node.js docs for `node --test` and `Object.freeze`.

SUBMISSION

# Deliverables (all included in this PDF) and checklist

| Required deliverable | Where |
|---|---|
| Source code — initial and final versions | Appendix B, C (p. 17–24) · Git `48bd593` → `a78e2e9` |
| README with setup, run and test commands | Appendix A (p. 17) |
| Test files and actual execution results | Appendix D, E · slides 11–12 |
| Analysis / refactoring report, diagrams, quality comparison | Slides 3–13 |
| AI evidence appendix | Slides 5, 14, 15 |

| Checklist item | Status |
|---|---|
| Existing availability rules still work | ✔ **AC1–AC6 tests** |
| At least four fictional rooms | ✔ **5 rooms** |
| Capacity and equipment filters work | ✔ **F1/F2 tests** |
| At least ten executable tests | ✔ **25 (+8 baseline)** |
| Refactoring and evaluation evidence | ✔ **slides 9–13** |
| AI use and human review documented | ✔ **slides 5, 14, 15** |

Run locally: `npm install` · `npm test` · `npm start` → `localhost:8080/final/`

# APPENDIX · SOURCE CODE, TESTS AND RESULTS

## A. README.md

```
# Smart Campus Room Availability Checker — Assignment #04

Author: Yeraliyeva Anel (24B031020) · Software Engineering, KBTU · Practice: Tuesday 18:00–19:00

AI-assisted evolution of the Assignment #03 single-room prototype into a multi-room checker with
capacity/equipment filtering, a refactored module structure and an automated test suite.
Fictional data only. No login, payments, live APIs or database. Fixed current time: **2026-11-10 09:00**.

## Layout

```
baseline/               original single-room prototype (initial version)
  index.html, app.js    data + rules + DOM in one file
  tests/baseline.test.js  AC1–AC5 regression checks on the original
final/                  refactored multi-room version (final version)
  index.html
  src/data/rooms.js         room catalog (5 rooms, deep-frozen) + fixed NOW
  src/domain/time.js        time parsing + overlap rule (single source of truth)
  src/domain/availability.js  AC1–AC4 rules for ONE room (pure)
  src/domain/filters.js     F1–F3 filtering (pure)
  src/ui/render.js          DOM rendering only
  src/ui/app.js             controller: wires form events to domain + render
  tests/                    availability, filters, ui (jsdom) suites
docs/evidence/          saved test outputs and mutation-check results
```

## Setup

Requires Node.js 18+ (tested with v22) and Python 3 (only for the static server).

```bash
npm install           # installs jsdom (dev dependency for the UI test)
```
```

## A. README.md (cont.)

```
## Run

ES modules need to be served over HTTP (not file://):

```bash
npm start             # python3 -m http.server 8080
# open http://localhost:8080/final/      (refactored version)
# open http://localhost:8080/baseline/   (original version)
```

## Test

```bash
npm run test:baseline  # 8 AC regression tests against the original prototype
npm test               # 25 tests against the final version (AC1–AC6, F1–F5, UI integration)
```

Results recorded on 2026-11-08: baseline **8/8 passed**, final **25/25 passed**
(full output in `docs/evidence/`).

## Git history

```bash
git log --oneline      # baseline commit first, final refactor after it
```

## Acceptance criteria

| ID | Rule |
|----|------|
| AC1 | Valid HH:MM times, end after start, start not before the fixed current time |
| AC2 | Maximum 120 minutes (120 allowed, 121 rejected) |
| AC3 | Blocked room cannot be booked |
| AC4 | Half-open intervals: touching bookings allowed, overlapping rejected |
| AC5 | Booking data is never mutated (catalog is deep-frozen) |
| AC6 | Clear, responsive feedback (status region, mobile viewport) |
```

# APPENDIX · SOURCE CODE, TESTS AND RESULTS

### A. README.md (cont.)

```
| F1 | Capacity ≥ attendees |
| F2 | Room has ALL selected equipment |
| F3 | Blocked room never reported available (shown with BLOCKED label) |
| F4 | Clear empty state when no rooms match |
| F5 | Changing the selected room refreshes the result with that room's bookings
 ↳ |
```

### A. package.json

```
{
  "name": "smart-campus-room-checker",
  "version": "2.0.0",
  "private": true,
  "scripts": {
    "test:baseline": "node --test --test-reporter=spec baseline/tests/*.test.js",
    "test": "node --test --test-reporter=spec final/tests/*.test.js",
    "start": "python3 -m http.server 8080"
  },
  "devDependencies": {
    "jsdom": "^24.1.3"
  }
}
```

### B. Initial version — baseline/app.js

```
// Smart Campus Room Availability Checker — BASELINE (Assignment #03 / Practice #
 ↳ 06 style)
// Single room, single file: data, rules and DOM code are mixed together.

var NOW = "2026-11-10T09:00"; // fixed "current time" for the prototype

var room = {
  name: "Room 301",
  blocked: false,
  bookings: [
    { start: "2026-11-10T10:00", end: "2026-11-10T11:00", title: "Algorithms lect
 ↳ ure" },
    { start: "2026-11-10T13:00", end: "2026-11-10T14:30", title: "Club meeting" }
  ]
};
```

### B. Initial version — baseline/app.js (cont.)

```

function checkAvailability(date, startTime, endTime, r) {
  r = r || room;
  // AC1: valid time
  if (!/^\d{2}:\d{2}$/.test(startTime) || !/^\d{2}:\d{2}$/.test(endTime)) {
    return { ok: false, message: "Please enter a valid start and end time." };
  }
  var s = new Date(date + "T" + startTime).getTime();
  var e = new Date(date + "T" + endTime).getTime();
  if (isNaN(s) || isNaN(e)) {
    return { ok: false, message: "Please enter a valid start and end time." };
  }
  if (e <= s) {
    return { ok: false, message: "End time must be after start time." };
  }
  if (s < new Date(NOW).getTime()) {
    return { ok: false, message: "Start time cannot be in the past." };
  }
  // AC2: 120 minute max
  if ((e - s) / 60000 > 120) {
    return { ok: false, message: "Bookings cannot be longer than 120 minutes." };
  }
  // AC3: blocked room
  if (r.blocked) {
    return { ok: false, message: r.name + " is blocked and cannot be booked." };
  }
  // AC4: overlap (touching intervals are allowed)
  for (var i = 0; i < r.bookings.length; i++) {
    var b = r.bookings[i];
    var bs = new Date(b.start).getTime();
    var be = new Date(b.end).getTime();
    if (s < be && bs < e) {
      return { ok: false, message: "Conflicts with \"" + b.title + "\"." };
    }
  }
  return { ok: true, message: r.name + " is available." };
}

```

### B. Initial version — baseline/app.js (cont.)

```
// ---------- UI (duplicated validation + rendering in two handlers) ----------
if (typeof document !== "undefined") {
  document.addEventListener("DOMContentLoaded", function () {
    var form = document.getElementById("form");
    var out = document.getElementById("result");
    document.getElementById("roomName").textContent = room.name;

    form.addEventListener("submit", function (ev) {
      ev.preventDefault();
      var d = document.getElementById("date").value;
      var st = document.getElementById("start").value;
      var en = document.getElementById("end").value;
      // duplicated quick validation (same as AC1 in checkAvailability)
      if (!/^\d{2}:\d{2}$/.test(st) || !/^\d{2}:\d{2}$/.test(en)) {
        out.className = "result error";
        out.textContent = "Please enter a valid start and end time.";
        return;
      }
      var res = checkAvailability(d, st, en);
      out.className = res.ok ? "result ok" : "result error";
      out.textContent = res.message;
    });

    form.addEventListener("change", function () {
      var d = document.getElementById("date").value;
      var st = document.getElementById("start").value;
      var en = document.getElementById("end").value;
      // duplicated quick validation again
      if (!/^\d{2}:\d{2}$/.test(st) || !/^\d{2}:\d{2}$/.test(en)) {
        out.className = "result";
        out.textContent = "";
        return;
      }
      var res = checkAvailability(d, st, en);
      out.className = res.ok ? "result ok" : "result error";
      out.textContent = res.message;
    });
  });
```

### B. Initial version — baseline/app.js (cont.)

```
}

if (typeof module !== "undefined") {
  module.exports = { checkAvailability: checkAvailability, room: room, NOW: NOW }
;
}
```

### B. Initial version — baseline/index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Room Checker (baseline)</title>
<style>
  body { font-family: system-ui, sans-serif; max-width: 480px; margin: 2rem auto;
 padding: 0 16px; }
  label { display: block; margin-top: .75rem; }
  input, button { width: 100%; padding: .5rem; font-size: 1rem; box-sizing: borde
r-box; }
  button { margin-top: 1rem; }
  .result { margin-top: 1rem; padding: .75rem; border-radius: 6px; }
  .ok { background: #e3f6e8; color: #145c2a; }
  .error { background: #fde8e8; color: #8a1c1c; }
</style>
</head>
<body>
  <h1>Is <span id="roomName"></span> free?</h1>
  <p>Current time (fixed): 10 Nov 2026, 09:00</p>
  <form id="form">
    <label>Date <input id="date" type="date" value="2026-11-10"></label>
    <label>Start <input id="start" type="time"></label>
    <label>End <input id="end" type="time"></label>
    <button type="submit">Check availability</button>
  </form>
  <div id="result" class="result" role="status" aria-live="polite"></div>
  <script src="app.js"></script>
</body>
```

### B. Initial version — baseline/index.html (cont.)

```
</html>
```

### B. Initial version — baseline/tests/baseline.test.js

```
// Baseline regression suite: AC1–AC5 run against the ORIGINAL single-room prototype.
const test = require("node:test");
const assert = require("node:assert/strict");
const { checkAvailability, room } = require("../app.js");

const D = "2026-11-10";

test("AC1: rejects end before start", () => {
  assert.equal(checkAvailability(D, "12:00", "11:00").ok, false);
});

test("AC1: rejects start in the past (before fixed now 09:00)", () => {
  assert.equal(checkAvailability(D, "08:00", "08:30").ok, false);
});

test("AC2: exactly 120 minutes is allowed", () => {
  assert.equal(checkAvailability(D, "15:00", "17:00").ok, true);
});

test("AC2: 121 minutes is rejected", () => {
  assert.equal(checkAvailability(D, "15:00", "17:01").ok, false);
});

test("AC3: blocked room is never available", () => {
  const blocked = { ...room, blocked: true };
  assert.equal(checkAvailability(D, "15:00", "16:00", blocked).ok, false);
});

test("AC4: touching interval (11:00-12:00 after 10:00-11:00) is allowed", () => {
  assert.equal(checkAvailability(D, "11:00", "12:00").ok, true);
});

test("AC4: overlapping interval (10:30-11:30) is rejected", () => {
  assert.equal(checkAvailability(D, "10:30", "11:30").ok, false);
```

### B. Initial version — baseline/tests/baseline.test.js (cont.)

```
});

test("AC5: checking availability does not mutate booking data", () => {
  const before = JSON.stringify(room.bookings);
  checkAvailability(D, "10:30", "11:30");
  checkAvailability(D, "15:00", "16:00");
  assert.equal(JSON.stringify(room.bookings), before);
});
```

### C. Final version — final/src/data/rooms.js

```
// Room catalog — fictional data only. Deep-frozen so no code path can mutate bookings (AC5).
export const NOW = "2026-11-10T09:00"; // fixed current time

export const EQUIPMENT = Object.freeze(["projector", "whiteboard", "video-conf", "computers"]);

const deepFreeze = (obj) => {
  Object.values(obj).forEach((v) => { if (v && typeof v === "object") deepFreeze(v); });
  return Object.freeze(obj);
};

export const ROOMS = deepFreeze([
  {
    id: "R301", name: "Room 301", capacity: 30, blocked: false,
    equipment: ["projector", "whiteboard"],
    bookings: [
      { start: "2026-11-10T10:00", end: "2026-11-10T11:00", title: "Algorithms lecture" },
      { start: "2026-11-10T13:00", end: "2026-11-10T14:30", title: "Club meeting" }
    ]
  },
  {
    id: "R105", name: "Seminar Room 105", capacity: 12, blocked: false,
    equipment: ["whiteboard", "video-conf"],
    bookings: [
```

### C. Final version — final/src/data/rooms.js (cont.)

```
24        { start: "2026-11-10T11:00", end: "2026-11-10T12:00", title: "Thesis defens
  ↳ e rehearsal" }
25      ]
26    },
27    {
28      id: "LAB2", name: "Computer Lab 2", capacity: 24, blocked: false,
29      equipment: ["projector", "computers", "whiteboard"],
30      bookings: [
31        { start: "2026-11-10T09:00", end: "2026-11-10T10:30", title: "Programming p
  ↳ ractice" }
32      ]
33    },
34    {
35      id: "AUD1", name: "Main Auditorium", capacity: 120, blocked: true,
36      equipment: ["projector", "video-conf"],
37      bookings: []
38    },
39    {
40      id: "R210", name: "Study Room 210", capacity: 6, blocked: false,
41      equipment: ["whiteboard"],
42      bookings: []
43    }
44  ]);
45
46  export const findRoom = (rooms, id) => rooms.find((r) => r.id === id) || null;
```

### C. Final version — final/src/domain/time.js

```
 1  // Single source of truth for time parsing (previously duplicated in 3 places).
 2  const HHMM = /^([01]\d|2[0-3]):[0-5]\d$/;
 3
 4  export const isValidHHMM = (value) => typeof value === "string" && HHMM.test(valu
  ↳ e);
 5
 6  /** Returns epoch ms for "YYYY-MM-DD" + "HH:MM", or NaN. */
 7  export const toMillis = (date, hhmm) => new Date(`${date}T${hhmm}`).getTime();
 8
 9  export const minutesBetween = (startMs, endMs) => (endMs - startMs) / 60000;
10
```

### C. Final version — final/src/domain/time.js (cont.)

```
11  /** Half-open intervals [aStart,aEnd) and [bStart,bEnd): touching is NOT overlap.
  ↳  */
12  export const overlaps = (aStart, aEnd, bStart, bEnd) => aStart < bEnd && bStart <
  ↳  aEnd;
```

### C. Final version — final/src/domain/availability.js

```
 1  // Booking / availability rules AC1–AC4. Pure functions: no DOM, no mutation (AC5
  ↳ ).
 2  import { isValidHHMM, toMillis, minutesBetween, overlaps } from "./time.js";
 3
 4  export const MAX_MINUTES = 120;
 5
 6  export const MESSAGES = Object.freeze({
 7    invalidTime: "Please enter a valid start and end time.",
 8    endBeforeStart: "End time must be after start time.",
 9    past: "Start time cannot be in the past.",
10    tooLong: `Bookings cannot be longer than ${MAX_MINUTES} minutes.`,
11    noRoom: "Please select a room first."
12  });
13
14  /** AC1 + AC2: validate the requested slot independently of any room. */
15  export function validateSlot({ date, start, end }, now) {
16    if (!isValidHHMM(start) || !isValidHHMM(end)) return { ok: false, reason: "inva
  ↳ lid", message: MESSAGES.invalidTime };
17    const s = toMillis(date, start);
18    const e = toMillis(date, end);
19    if (Number.isNaN(s) || Number.isNaN(e)) return { ok: false, reason: "invalid",
  ↳ message: MESSAGES.invalidTime };
20    if (e <= s) return { ok: false, reason: "order", message: MESSAGES.endBeforeSta
  ↳ rt };
21    if (s < new Date(now).getTime()) return { ok: false, reason: "past", message: M
  ↳ ESSAGES.past };
22    if (minutesBetween(s, e) > MAX_MINUTES) return { ok: false, reason: "duration",
  ↳  message: MESSAGES.tooLong };
23    return { ok: true, startMs: s, endMs: e };
24  }
25
26  /** AC1–AC4 for ONE room, using only that room's bookings. */
```

### C. Final version — final/src/domain/availability.js (cont.)

```
27  export function checkAvailability(room, slot, now) {
28    if (!room) return { ok: false, reason: "no-room", message: MESSAGES.noRoom };
29    const v = validateSlot(slot, now);
30    if (!v.ok) return v;
31    if (room.blocked) return { ok: false, reason: "blocked", message: `${room.name}
  ↳  is blocked and cannot be booked.` };
32    const conflict = room.bookings.find((b) =>
33      overlaps(v.startMs, v.endMs, new Date(b.start).getTime(), new Date(b.end).get
  ↳ Time()));
34    if (conflict) return { ok: false, reason: "conflict", message: `${room.name} co
  ↳ nflicts with "${conflict.title}" (${conflict.start.slice(11)}–${conflict.end.slic
  ↳ e(11)}).` };
35    return { ok: true, reason: "available", message: `${room.name} is available.` }
  ↳ ;
36  }
```

### C. Final version — final/src/domain/filters.js

```
 1  // Room filtering rules F1–F3. Pure functions.
 2
 3  /** F1: capacity >= attendees. Empty/invalid attendees means "no capacity require
  ↳ ment". */
 4  export const meetsCapacity = (room, attendees) => {
 5    const n = Number(attendees);
 6    return !Number.isFinite(n) || n <= 0 || room.capacity >= n;
 7  };
 8
 9  /** F2: room must contain ALL required items. */
10  export const hasAllEquipment = (room, required = []) => required.every((item) =>
  ↳ room.equipment.includes(item));
11
12  /**
13   * Rooms matching the user's requirements. Blocked rooms stay in the list but are
14   * flagged (F3: they can never be reported as available — see checkAvailability).
15   */
16  export function filterRooms(rooms, { attendees, equipment = [] } = {}) {
17    return rooms.filter((r) => meetsCapacity(r, attendees) && hasAllEquipment(r, eq
  ↳ uipment));
18  }
```

### C. Final version — final/src/ui/render.js

```
 1  // UI presentation only: turns state into DOM. One renderResult() replaces the tw
  ↳ o copies in the baseline.
 2
 3  export function renderRoomList(listEl, rooms, selectedId) {
 4    listEl.replaceChildren();
 5    for (const room of rooms) {
 6      const li = listEl.ownerDocument.createElement("li");
 7      const label = listEl.ownerDocument.createElement("label");
 8      label.className = "room" + (room.blocked ? " blocked" : "");
 9      const radio = listEl.ownerDocument.createElement("input");
10      radio.type = "radio";
11      radio.name = "room";
12      radio.value = room.id;
13      radio.checked = room.id === selectedId;
14      const info = listEl.ownerDocument.createElement("span");
15      info.innerHTML = `<strong></strong> <small></small>`;
16      info.querySelector("strong").textContent = room.name;
17      info.querySelector("small").textContent =
18        `${room.capacity} seats · ${room.equipment.join(", ")}${room.blocked ? " ·
  ↳ BLOCKED" : ""}`;
19      label.append(radio, info);
20      li.append(label);
21      listEl.append(li);
22    }
23  }
24
25  export function renderEmptyState(emptyEl, isEmpty) {
26    emptyEl.hidden = !isEmpty;
27  }
28
29  export function renderResult(resultEl, result) {
30    if (!result) {
31      resultEl.className = "result";
32      resultEl.textContent = "";
33      return;
34    }
35    resultEl.className = `result ${result.ok ? "ok" : "error"}`;
36    resultEl.dataset.reason = result.reason;
```

### C. Final version — final/src/ui/render.js (cont.)

```
37    resultEl.textContent = result.message;
38  }
```

### C. Final version — final/src/ui/app.js

```
 1  // UI controller: reads the form, calls domain functions, delegates drawing to re
  ↳ nder.js.
 2  import { ROOMS, NOW, findRoom } from "../data/rooms.js";
 3  import { filterRooms } from "../domain/filters.js";
 4  import { checkAvailability } from "../domain/availability.js";
 5  import { renderRoomList, renderEmptyState, renderResult } from "./render.js";
 6
 7  export function mountApp(doc, { rooms = ROOMS, now = NOW } = {}) {
 8    const $ = (id) => doc.getElementById(id);
 9    const form = $("form"), list = $("rooms"), empty = $("empty"), result = $("resu
  ↳ lt");
10    const state = { selectedId: null, checked: false };
11
12    const readCriteria = () => ({
13      attendees: $("attendees").value,
14      equipment: [...doc.querySelectorAll('input[name="equipment"]:checked')].map((
  ↳ c) => c.value)
15    });
16    const readSlot = () => ({ date: $("date").value, start: $("start").value, end:
  ↳ $("end").value });
17
18    function refresh() {
19      const visible = filterRooms(rooms, readCriteria());
20      // selected room filtered out -> drop selection so we never show a stale resu
  ↳ lt
21      if (!visible.some((r) => r.id === state.selectedId)) state.selectedId = null;
22      renderRoomList(list, visible, state.selectedId);
23      renderEmptyState(empty, visible.length === 0);
24      // F5: result always reflects the currently selected room
25      if (!state.checked) return renderResult(result, null);
26      const room = findRoom(rooms, state.selectedId);
27      renderResult(result, checkAvailability(room, readSlot(), now));
28    }
29
```

### C. Final version — final/src/ui/app.js (cont.)

```
30    list.addEventListener("change", (ev) => {
31      if (ev.target.name === "room") { state.selectedId = ev.target.value; refresh(
  ↳ ); }
32    });
33    form.addEventListener("input", (ev) => {
34      if (ev.target.name !== "room") refresh(); // filters/time edits re-render imm
  ↳ ediately
35    });
36    form.addEventListener("submit", (ev) => {
37      ev.preventDefault();
38      state.checked = true;
39      refresh();
40    });
41
42    refresh();
43    return { state, refresh };
44  }
```

### C. Final version — final/index.html

```
 1  <!doctype html>
 2  <html lang="en">
 3  <head>
 4  <meta charset="utf-8">
 5  <meta name="viewport" content="width=device-width, initial-scale=1">
 6  <title>Smart Campus Room Checker</title>
 7  <link rel="stylesheet" href="src/ui/styles.css">
 8  </head>
 9  <body>
10  <main>
11    <h1>Smart Campus Room Checker</h1>
12    <p class="now">Current time (fixed): 10 Nov 2026, 09:00</p>
13    <form id="form" novalidate>
14      <fieldset>
15        <legend>1. Requirements</legend>
16        <label>Attendees <input id="attendees" type="number" min="1" inputmode="num
  ↳ eric" placeholder="e.g. 20"></label>
17        <div class="equipment">
18          <label><input type="checkbox" name="equipment" value="projector"> Project
```

### C. Final version — final/index.html (cont.)

```
  ↳ or</label>
19          <label><input type="checkbox" name="equipment" value="whiteboard"> Whiteb
  ↳ oard</label>
20          <label><input type="checkbox" name="equipment" value="video-conf"> Video
  ↳ conferencing</label>
21          <label><input type="checkbox" name="equipment" value="computers"> Compute
  ↳ rs</label>
22        </div>
23      </fieldset>
24      <fieldset>
25        <legend>2. Choose a room</legend>
26        <ul id="rooms" class="rooms"></ul>
27        <p id="empty" class="empty" hidden>No rooms match these requirements. Try f
  ↳ ewer attendees or less equipment.</p>
28      </fieldset>
29      <fieldset>
30        <legend>3. Time slot</legend>
31        <div class="row">
32          <label>Date <input id="date" type="date" value="2026-11-10"></label>
33          <label>Start <input id="start" type="time"></label>
34          <label>End <input id="end" type="time"></label>
35        </div>
36      </fieldset>
37      <button type="submit">Check availability</button>
38    </form>
39    <div id="result" class="result" role="status" aria-live="polite"></div>
40  </main>
41  <script type="module">
42    import { mountApp } from "./src/ui/app.js";
43    mountApp(document);
44  </script>
45  </body>
46  </html>
```

### C. Final version — final/src/ui/styles.css

```
 1  * { box-sizing: border-box; }
 2  body { font-family: system-ui, sans-serif; margin: 0; background: #f6f7f9; color:
  ↳  #1d2330; }
 3  main { max-width: 640px; margin: 0 auto; padding: 24px 16px; }
 4  h1 { font-size: 1.5rem; margin: 0 0 .25rem; }
 5  .now { color: #5b6475; margin-top: 0; }
 6  fieldset { border: 1px solid #d9dde5; border-radius: 10px; background: #fff; marg
  ↳ in: 0 0 16px; padding: 12px 16px; }
 7  legend { font-weight: 600; padding: 0 4px; }
 8  label { display: block; margin: 6px 0; }
 9  input[type=number], input[type=date], input[type=time] { width: 100%; padding: 8p
  ↳ x; font-size: 1rem; border: 1px solid #c4cad6; border-radius: 6px; }
10  .equipment { display: grid; grid-template-columns: repeat(auto-fit, minmax(160px,
  ↳  1fr)); }
11  .rooms { list-style: none; padding: 0; margin: 0; }
12  .room { display: flex; gap: 10px; align-items: flex-start; padding: 8px; border-r
  ↳ adius: 6px; cursor: pointer; }
13  .room:hover { background: #f0f3f8; }
14  .room small { display: block; color: #5b6475; }
15  .room.blocked { opacity: .7; }
16  .row { display: grid; grid-template-columns: repeat(auto-fit, minmax(150px, 1fr))
  ↳ ; gap: 8px; }
17  .empty { color: #8a5a00; background: #fff6e0; padding: 10px; border-radius: 6px;
  ↳ }
18  button { width: 100%; padding: 12px; font-size: 1rem; border: 0; border-radius: 8
  ↳ px; background: #2950c8; color: #fff; cursor: pointer; }
19  .result { margin-top: 16px; padding: 12px; border-radius: 8px; }
20  .result:empty { display: none; }
21  .ok { background: #e3f6e8; color: #145c2a; }
22  .error { background: #fde8e8; color: #8a1c1c; }
```

### D. Tests — final/tests/availability.test.js

```
1  // Regression tests AC1–AC5 against the REFACTORED domain module.
2  import test from "node:test";
3  import assert from "node:assert/strict";
4  import { checkAvailability } from "../src/domain/availability.js";
5  import { ROOMS, NOW, findRoom } from "../src/data/rooms.js";
6
```

### D. Tests — final/tests/availability.test.js (cont.)

```
 7  const R301 = findRoom(ROOMS, "R301"); // bookings 10:00–11:00, 13:00–14:30
 8  const slot = (start, end) => ({ date: "2026-11-10", start, end });
 9
10  test("AC1: rejects malformed time", () => {
11    assert.equal(checkAvailability(R301, slot("9am", "10:00"), NOW).reason, "invali
    ↳ d");
12  });
13  test("AC1: rejects end before start", () => {
14    assert.equal(checkAvailability(R301, slot("12:00", "11:00"), NOW).reason, "orde
    ↳ r");
15  });
16  test("AC1: rejects start before fixed current time (09:00)", () => {
17    assert.equal(checkAvailability(R301, slot("08:00", "08:30"), NOW).reason, "past
    ↳ ");
18  });
19  test("AC2 boundary: exactly 120 minutes is allowed", () => {
20    assert.equal(checkAvailability(R301, slot("15:00", "17:00"), NOW).ok, true);
21  });
22  test("AC2 boundary: 121 minutes is rejected", () => {
23    assert.equal(checkAvailability(R301, slot("15:00", "17:01"), NOW).reason, "dura
    ↳ tion");
24  });
25  test("AC3/F3: blocked room is never reported available, even with a free slot", (
    ↳ ) => {
26    const r = checkAvailability(findRoom(ROOMS, "AUD1"), slot("15:00", "16:00"), NO
    ↳ W);
27    assert.equal(r.ok, false);
28    assert.equal(r.reason, "blocked");
29  });
30  test("AC4 boundary: touching interval 11:00–12:00 after 10:00–11:00 is allowed",
    ↳ () => {
31    assert.equal(checkAvailability(R301, slot("11:00", "12:00"), NOW).ok, true);
32  });
33  test("AC4 boundary: overlapping interval 10:59–11:30 is rejected", () => {
34    assert.equal(checkAvailability(R301, slot("10:59", "11:30"), NOW).reason, "conf
    ↳ lict");
35  });
36  test("AC4: interval fully containing a booking is rejected", () => {
```

### D. Tests — final/tests/availability.test.js (cont.)

```
37    assert.equal(checkAvailability(R301, slot("12:45", "14:45"), NOW).reason, "conf
    ↳ lict");
38  });
39  test("AC5: booking data is immutable (frozen) and unchanged after checks", () =>
    ↳ {
40    const before = JSON.stringify(ROOMS);
41    checkAvailability(R301, slot("10:30", "11:30"), NOW);
42    checkAvailability(R301, slot("15:00", "16:00"), NOW);
43    assert.equal(JSON.stringify(ROOMS), before);
44    assert.throws(() => { "use strict"; R301.bookings.push({}); }, TypeError);
45  });
46  test("Room-specific data: same slot is busy in R301 but free in R210", () => {
47    const s = slot("10:00", "11:00");
48    assert.equal(checkAvailability(R301, s, NOW).ok, false);
49    assert.equal(checkAvailability(findRoom(ROOMS, "R210"), s, NOW).ok, true);
50  });
51  test("No room selected gives a clear message instead of a crash", () => {
52    assert.equal(checkAvailability(null, slot("15:00", "16:00"), NOW).reason, "no-r
    ↳ oom");
53  });
```

### D. Tests — final/tests/filters.test.js

```
 1  // New feature tests F1–F3 (pure filtering logic).
 2  import test from "node:test";
 3  import assert from "node:assert/strict";
 4  import { filterRooms } from "../src/domain/filters.js";
 5  import { ROOMS } from "../src/data/rooms.js";
 6
 7  const ids = (rooms) => rooms.map((r) => r.id).sort();
 8
 9  test("Catalog has at least four rooms with id, name, capacity, equipment, blocked
    ↳ , bookings", () => {
10    assert.ok(ROOMS.length >= 4);
11    for (const r of ROOMS) {
12      assert.equal(typeof r.id, "string");
13      assert.equal(typeof r.name, "string");
14      assert.equal(typeof r.capacity, "number");
15      assert.ok(Array.isArray(r.equipment));
```

### D. Tests — final/tests/filters.test.js (cont.)

```
16      assert.equal(typeof r.blocked, "boolean");
17      assert.ok(Array.isArray(r.bookings));
18    }
19  });
20  test("F1 boundary: exact capacity (12) keeps Seminar Room 105", () => {
21    assert.ok(ids(filterRooms(ROOMS, { attendees: 12 })).includes("R105"));
22  });
23  test("F1 boundary: insufficient capacity (13) removes Seminar Room 105", () => {
24    assert.ok(!ids(filterRooms(ROOMS, { attendees: 13 })).includes("R105"));
25  });
26  test("F1: empty attendees means no capacity restriction", () => {
27    assert.equal(filterRooms(ROOMS, { attendees: "" }).length, ROOMS.length);
28  });
29  test("F2: room must contain ALL selected equipment (projector + computers -> LAB2
  ↳  only)", () => {
30    assert.deepEqual(ids(filterRooms(ROOMS, { equipment: ["projector", "computers"]
  ↳  })), ["LAB2"]);
31  });
32  test("F2: one missing item excludes the room (whiteboard + video-conf -> R105 onl
  ↳ y)", () => {
33    assert.deepEqual(ids(filterRooms(ROOMS, { equipment: ["whiteboard", "video-conf
  ↳ "] })), ["R105"]);
34  });
35  test("F1+F2 combined: 25 attendees + projector -> R301, AUD1 (blocked shown, flag
  ↳ ged)", () => {
36    const out = filterRooms(ROOMS, { attendees: 25, equipment: ["projector"] });
37    assert.deepEqual(ids(out), ["AUD1", "R301"]);
38    assert.equal(out.find((r) => r.id === "AUD1").blocked, true);
39  });
40  test("F4 (logic): impossible requirements return an empty list", () => {
41    assert.deepEqual(filterRooms(ROOMS, { attendees: 500 }), []);
42  });
```

### D. Tests — final/tests/ui.test.js

```
1  // Integration / UI behaviour checks: real index.html + app.js in jsdom.
2  import test from "node:test";
3  import assert from "node:assert/strict";
4  import { readFileSync } from "node:fs";
```

### D. Tests — final/tests/ui.test.js (cont.)

```
5  import { JSDOM } from "jsdom";
6  import { mountApp } from "../src/ui/app.js";
7
8  const html = readFileSync(new URL("../index.html", import.meta.url), "utf8")
9    .replace(/<script type="module">[\s\S]*?<\/script>/, ""); // mount manually bel
  ↳ ow
10
11  function setup() {
12    const dom = new JSDOM(html);
13    const doc = dom.window.document;
14    mountApp(doc);
15    const fire = (el, type) => el.dispatchEvent(new dom.window.Event(type, { bubble
  ↳ s: true }));
16    const set = (id, v) => { doc.getElementById(id).value = v; fire(doc.getElementB
  ↳ yId(id), "input"); };
17    const pick = (roomId) => { const r = doc.querySelector(`input[name=room][value=
  ↳ ${roomId}]`); r.checked = true; fire(r, "change"); };
18    const submit = () => doc.getElementById("form").dispatchEvent(new dom.window.Ev
  ↳ ent("submit", { cancelable: true }));
19    const result = () => doc.getElementById("result");
20    const roomIds = () => [...doc.querySelectorAll("input[name=room]")].map((r) =>
  ↳ r.value);
21    return { doc, set, pick, submit, result, roomIds, fire };
22  }
23
24  test("UI: successful availability check shows green 'available' feedback", () =>
  ↳ {
25    const ui = setup();
26    ui.pick("R210"); ui.set("start", "11:00"); ui.set("end", "12:00"); ui.submit();
27    assert.equal(ui.result().className, "result ok");
28    assert.match(ui.result().textContent, /Study Room 210 is available/);
29  });
30
31  test("F5 (UI): switching room refreshes result using the new room's bookings", ()
  ↳  => {
32    const ui = setup();
33    ui.pick("R210"); ui.set("start", "10:00"); ui.set("end", "11:00"); ui.submit();
34    assert.equal(ui.result().dataset.reason, "available");
```

### D. Tests — final/tests/ui.test.js (cont.)

```
35    ui.pick("R301"); // R301 has a 10:00–11:00 lecture
36    assert.equal(ui.result().dataset.reason, "conflict");
37    assert.match(ui.result().textContent, /Algorithms lecture/);
38  });
39
40  test("F4 (UI): clear empty state when no rooms match; selection and result cleare
  ↳ d", () => {
41    const ui = setup();
42    ui.pick("R301"); ui.set("start", "15:00"); ui.set("end", "16:00"); ui.submit();
43    assert.equal(ui.result().dataset.reason, "available");
44    ui.set("attendees", "500");
45    assert.equal(ui.roomIds().length, 0);
46    assert.equal(ui.doc.getElementById("empty").hidden, false);
47    assert.equal(ui.result().dataset.reason, "no-room");
48  });
49
50  test("F1/F2 (UI): filters update the visible room list", () => {
51    const ui = setup();
52    assert.equal(ui.roomIds().length, 5);
53    const cb = ui.doc.querySelector('input[name=equipment][value=computers]');
54    cb.checked = true; ui.fire(cb, "input");
55    assert.deepEqual(ui.roomIds(), ["LAB2"]);
56  });
57
58  test("AC6 (UI): invalid slot gives clear error text; layout has responsive viewpo
  ↳ rt", () => {
59    const ui = setup();
60    ui.pick("R301"); ui.set("start", "15:00"); ui.set("end", "17:30"); ui.submit();
61    assert.equal(ui.result().className, "result error");
62    assert.match(ui.result().textContent, /120 minutes/);
63    assert.match(ui.doc.querySelector('meta[name=viewport]').content, /width=device
  ↳ -width/);
64  });
```

### E. Test results — npm run test:baseline (actual output)

```
 1  ✔ AC1: rejects end before start (0.80313ms)
 2  ✔ AC1: rejects start in the past (before fixed now 09:00) (0.089563ms)
 3  ✔ AC2: exactly 120 minutes is allowed (0.101588ms)
 4  ✔ AC2: 121 minutes is rejected (0.066285ms)
 5  ✔ AC3: blocked room is never available (0.068814ms)
 6  ✔ AC4: touching interval (11:00-12:00 after 10:00-11:00) is allowed (0.060961ms)
 7  ✔ AC4: overlapping interval (10:30-11:30) is rejected (0.096111ms)
 8  ✔ AC5: checking availability does not mutate booking data (0.188559ms)
 9  ℹ tests 8
10  ℹ suites 0
11  ℹ pass 8
12  ℹ fail 0
13  ℹ cancelled 0
14  ℹ skipped 0
15  ℹ todo 0
16  ℹ duration_ms 47.269087
```

### E. Test results — npm test (actual output)

```
 1
 2  > smart-campus-room-checker@2.0.0 test
 3  > node --test --test-reporter=spec final/tests/*.test.js
 4
 5  ✔ AC1: rejects malformed time (1.562192ms)
 6  ✔ AC1: rejects end before start (0.276426ms)
 7  ✔ AC1: rejects start before fixed current time (09:00) (0.077566ms)
 8  ✔ AC2 boundary: exactly 120 minutes is allowed (0.09723ms)
 9  ✔ AC2 boundary: 121 minutes is rejected (0.070816ms)
10  ✔ AC3/F3: blocked room is never reported available, even with a free slot (0.0803
  ↳ 09ms)
11  ✔ AC4 boundary: touching interval 11:00–12:00 after 10:00–11:00 is allowed (0.059
  ↳ 021ms)
12  ✔ AC4 boundary: overlapping interval 10:59–11:30 is rejected (0.058364ms)
13  ✔ AC4: interval fully containing a booking is rejected (0.21555ms)
14  ✔ AC5: booking data is immutable (frozen) and unchanged after checks (0.447779ms)
15  ✔ Room-specific data: same slot is busy in R301 but free in R210 (0.103455ms)
16  ✔ No room selected gives a clear message instead of a crash (0.047375ms)
17  ✔ Catalog has at least four rooms with id, name, capacity, equipment, blocked, bo
  ↳ okings (1.280383ms)
```

### E. Test results — npm test (actual output) (cont.)

```
18  ✔ F1 boundary: exact capacity (12) keeps Seminar Room 105 (0.448635ms)
19  ✔ F1 boundary: insufficient capacity (13) removes Seminar Room 105 (1.560365ms)
20  ✔ F1: empty attendees means no capacity restriction (0.118358ms)
21  ✔ F2: room must contain ALL selected equipment (projector + computers -> LAB2 onl
  ↳ y) (0.542921ms)
22  ✔ F2: one missing item excludes the room (whiteboard + video-conf -> R105 only) (
  ↳ 0.107335ms)
23  ✔ F1+F2 combined: 25 attendees + projector -> R301, AUD1 (blocked shown, flagged)
  ↳  (0.090493ms)
24  ✔ F4 (logic): impossible requirements return an empty list (0.057142ms)
25  ✔ UI: successful availability check shows green 'available' feedback (62.566138ms
  ↳ )
26  ✔ F5 (UI): switching room refreshes result using the new room's bookings (24.1895
  ↳ 47ms)
27  ✔ F4 (UI): clear empty state when no rooms match; selection and result cleared (1
  ↳ 5.332797ms)
28  ✔ F1/F2 (UI): filters update the visible room list (11.144669ms)
29  ✔ AC6 (UI): invalid slot gives clear error text; layout has responsive viewport (
  ↳ 15.105089ms)
30  ℹ tests 25
31  ℹ suites 0
32  ℹ pass 25
33  ℹ fail 0
34  ℹ cancelled 0
35  ℹ skipped 0
36  ℹ todo 0
37  ℹ duration_ms 484.173971
```

### E. Mutation check 1 — overlaps() < → <=

```
1  ✖ AC4 boundary: touching interval 11:00–12:00 after 10:00–11:00 is allowed (0.619
 ↳ 781ms)
2  ℹ pass 24
3  ℹ fail 1
4  ✖ failing tests:
5  ✖ AC4 boundary: touching interval 11:00–12:00 after 10:00–11:00 is allowed (0.619
 ↳ 781ms)
```

### E. Mutation check 2 — every → some

```
1  ✖ F1 boundary: exact capacity (12) keeps Seminar Room 105 (7.928931ms)
2  ✖ F1: empty attendees means no capacity restriction (0.528013ms)
3  ✖ F2: room must contain ALL selected equipment (projector + computers -> LAB2 onl
 ↳ y) (1.041323ms)
4  ✖ F2: one missing item excludes the room (whiteboard + video-conf -> R105 only) (
 ↳ 0.371293ms)
5  ✖ UI: successful availability check shows green 'available' feedback (44.930247ms
 ↳ )
```

### E. Git history — git log --oneline

```
1  a78e2e9 Final: room catalog, F1-F5 filtering, refactor into data/domain/ui module
 ↳ s, 25 tests
2  535bb88 Baseline: fix test script
3  48bd593 Baseline: single-room prototype (Assignment #03) + AC regression tests
```